

Learning Augmented MPC for Attitude Control

Luca Byrnes

Motivation

Attitude (orientation) control is one of the most fundamental problems in spacecraft operation. Every subsystem onboard including solar panels, sensors, and propulsion depends on maintaining a certain orientation. Achieving this reliably is challenging because spacecraft follow rigid body dynamics, which are highly nonlinear. Despite these nonlinearities, the space industry's current gold standard remains Linear Quadratic Regulator (LQR) control. This controller relies on the system dynamics model to be accurate and locally linear, otherwise the performance degrades. What if we use a controller better equipped to handle nonlinearities? and that had the ability to learn model error?



System State

The state of the system is modeled through a hybrid approach, both using Euler angle error coordinates and quaternion representation. Quaternions are used to represent absolute rotation, such as the goal states or the current state, and Euler angle error coordinates are used to represent rotation error for use in the linearized models of the dynamics and are relative to some reference rotation quaternion.

$$x = \begin{bmatrix} q \\ \omega \end{bmatrix} \in R^7 \quad \bar{x} = \begin{bmatrix} \theta_{err} \\ \omega_{err} \end{bmatrix} \in R^6$$

This transformation is necessary as quaternions must maintain unit length to represent rotations, linearization does not guarantee this.

$$q_{err} = q_{goal}^{-1} \otimes q_{curr} \implies q_{err} = \begin{bmatrix} \cos\left(\frac{\|\theta_{err}\|}{2}\right) \\ \frac{\theta_{err}}{\|\theta_{err}\|} \sin\left(\frac{\|\theta_{err}\|}{2}\right) \end{bmatrix} \implies q_{err} \approx \begin{bmatrix} 1 \\ \frac{1}{2}\theta_{err} \end{bmatrix}$$

Methods

1. Spacecraft Dynamics Modeling

The spacecraft is modeled as a rigid body with rotational dynamics

$$\dot{q} = \frac{1}{2} \Omega(\omega) q$$

$$\dot{\omega} = I^{-1}(\tau - \omega \times (I\omega))$$

and simulated in discrete time using a RK4 integration scheme. Linear models for the dynamics were found using finite differencing, making finding derivatives easy, at the cost of computation.

$$x_{k+1} \approx A_d x_k + B_d u_k$$

2. Model Predictive Control

The MPC controller is implemented as a receding horizon multiple shooting trajectory optimization problem, where the dynamics are linearized around the nominal trajectory. This approach strikes a good middle ground between nonlinear modeling accuracy and computation speed.

$$\min_{\{\bar{x}_k, u_k\}} \sum_{k=0}^{N-1} (\bar{x}_k^T Q \bar{x}_k + u_k^T R u_k) + \bar{x}_N^T Q_f \bar{x}_N$$

$$\text{s.t. } \bar{x}_0 = \bar{x}_{init}$$

$$\bar{x}_{k+1} = A_d \bar{x}_k + B_d u_k$$

$$x_{min} \leq \bar{x}_k \leq x_{max}$$

$$u_{min} \leq u_k \leq u_{max}$$

3. Learning the Residual Dynamics

The residual dynamics can be defined as

$$r_\phi(x, u) \approx f_{true}(x, u) - f_{modeled}(x, u)$$

I used a neural network MLP to learn this function from a dataset generated by a closed loop simulation that had control input bias and perturbations to the inertia matrix. The dataset contained a list of entries each with the current state and input, predicted next state, and true next state. Using this data, I trained the NN offline using the current state and control as input, and the error in predicting the residual as the loss. The modeled and residual dynamics were then used in the linearized dynamics constraint for the Learning Augmented MPC controller.

$$x_{k+1} = f_{modeled}(x_k, u_k) + r_\phi(x_k, u_k)$$

Results

Under all cases the nominal MPC controller outperformed LQR in convergence, overshoot, and stability.

For low to moderate model error, where the residual dynamics makes up a minority of the total dynamics, The learning MPC outperforms the nominal MPC and LQR controllers.

Though at high model error the learning MPC has the worst performance and is unstable when LQR and nominal MPC remain stable. This is most likely caused by the jitteriness (non smoothness) found in the residual dynamics function, which can be seen in the control inputs.

Moderate Model Error Results

